Google Cloud Platform + AI Fundamentals

Universidad Austral
Facultad de Ingeniería
Primer Cuatrimestre 2026

Profesores
Mora Villa Abrille (BVilla@austral.edu.ar)
Rodrigo Pazos (RPazos@austral.edu.ar)

Configurando sistemas a mano

- Las arquitecturas de sistemas productivos pueden volverse complicadas muy rápidamente: redes, VMs, buckets, functions, etc. Llevar un registro de todos estos componentes y su estado es difícil.
- A la hora de actualizar esa arquitectura (sumar un componente, recomponer una parte, eliminarla) de forma manual hay varias desventajas: baja velocidad, posibilidad de errores, posible downtime extendido
- Lo ideal debería ser poder automatizar el *provisioning* la infraestructura. Asi nace la idea de Infrastructure as Code (IaC)
- Un script hecho por nosotros para levantar de forma sistemática servicios y componentes es una forma de armar IaC. Podemos correrlo varias veces, versionarlo, etc

Cloudformation y Terraform

- En 2011 AWS crea para si mismo un sistema llamado Cloudformation, que le permitía a los desarrolladores declarar, usando JSON, como debia verse la infraestructura de su proyecto
- A partir de esta declaración se podia replicar en cuestion de segundos y sin errores toda la infraestructura necesaria para un proyecto.

ElasticBeanstalkSample.template

```
1 {
2   "AWSTemplateFormatVersion" : "2010-09-09",
3
4   "Description" : "AWS CloudFormation Sample Template ElasticBeanstalkSample: Configure and
5
6   "Parameters" : {
7     "KeyName" : {
8       "Description" : "Name of an existing EC2 KeyPair to enable SSH access to the AWS Elast
9       "Type" : "String"
10    }
11  },
12
13  "Resources" : {
14    "sampleApplication" : {
15      "Type" : "AWS::ElasticBeanstalk::Application",
16      "Properties" : {
17        "Description" : "AWS Elastic Beanstalk Sample Application",
18        "ApplicationVersions" : [{
19          "VersionLabel" : "Initial Version",
20          "Description" : "Version 1.0",
21          "SourceBundle" : {
22            "S3Bucket" : { "Fn::Join" : ["-", ["elasticbeanstalk-samples", { "Ref" : "AWS::R
23            "S3Key" : "elasticbeanstalk-sampleapp.war"
24          }
25        }],
26        "ConfigurationTemplates" : [{
27          "TemplateName" : "DefaultConfiguration",
28          "Description" : "Default Configuration Version 1.0 - with SSH access",
29          "SolutionStackName" : "64bit Amazon Linux running Tomcat 7",
```

Cloudformation y Terraform

- En 2014 Hashicorp crea Terraform, una alternativa inicialmente open-source (hoy solo es source-available) para declarar infraestructura, extensible para funcionar con varios proveedores, no solo AWS.
- Tuvo un grado de adopción muy alto y hoy es prácticamente el estándar de la industrial

Code > DC_GCP > main.tf > provider "google"

```
provider "google" {
  project = "my-81bd"
  region  = "us-central1"
  zone    = "us-central1-c"
}

resource "google_compute_instance" "vm_instance" {
  name         = "terraform-instance"
  machine_type = "e2-micro"

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }

  network_interface {
    # A default network is created for all GCP projects
    network = "default"
    access_config {
    }
  }
}
```

Terraform CLI

- Terraform es una herramienta de tipo CLI que tiene 4 comandos principales
- `terraform init` se encarga de preparar el ambiente localmente, trayendo localmente informacion de modulos y proveedores, para poder usar esa información localmente para armar el plan
- `terraform plan` mira la declaración actual y la compara con el estado de nuestra infraestructura a través de la API. En función a eso arma un plan, que no es nada más que un archivo con varias entradas que explican todas las acciones que se van a aplicar en un orden específico. Hay 4 operaciones válidas: crear un recurso, actualizar un recurso, eliminar un recurso y reemplazar un recurso

Terraform CLI

- `plan` tambien se puede usar con la flag -out para guardar el plan en algun lado y aplicarlo más tarde.
- `apply` se encarga de aplicar un plan que le pasemos. Si no le pasamos nada crea un plan, nos lo muestra y pide confirmacion para aplicarlo.
- Por último `destroy` se encarga de elimiar los componentes de forma ordenada (primero las VMs, despues la VPC, porque al revés no es válido)

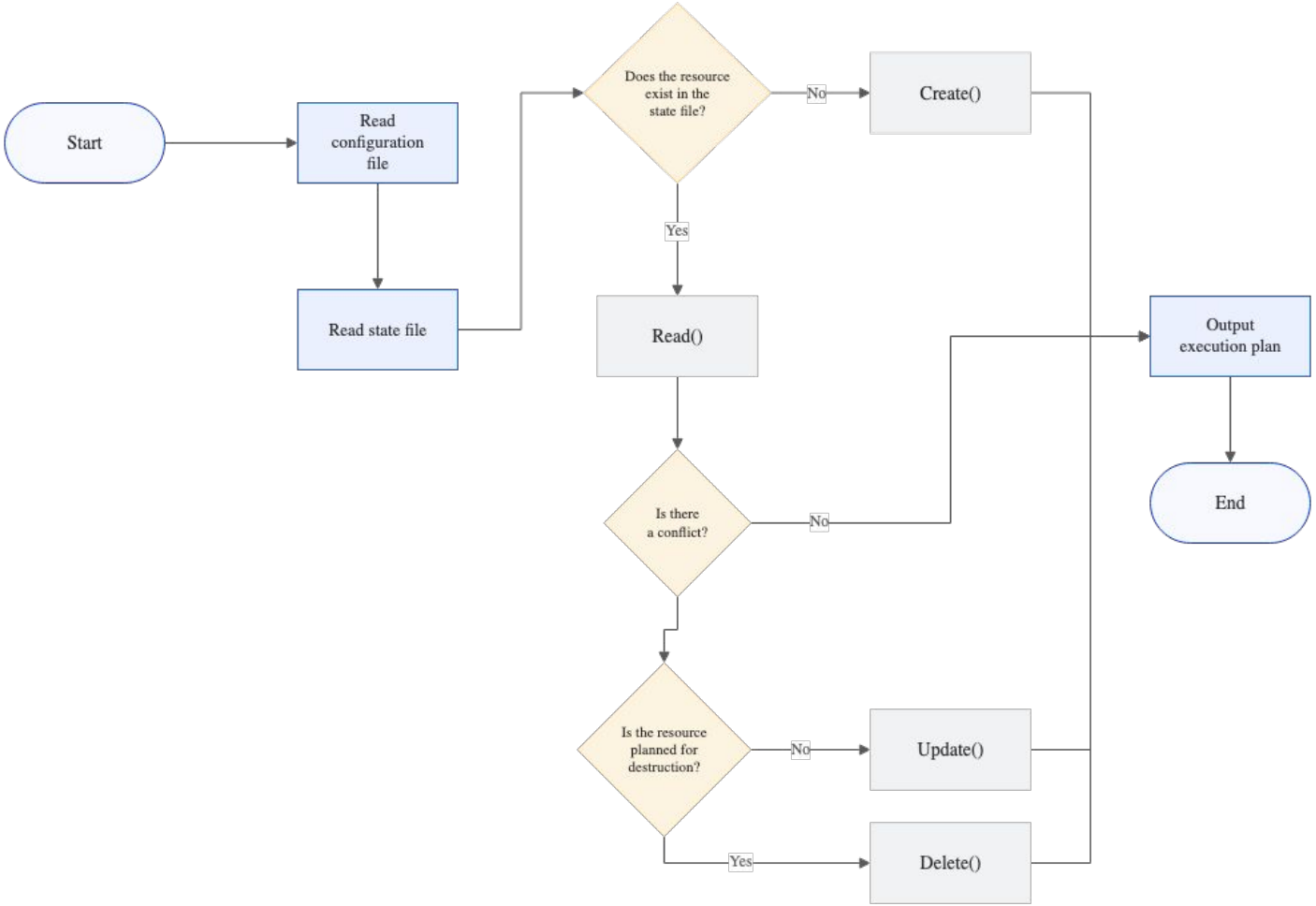
Terraform tf file

Terraform Plan

- Terraform tiene la inteligencia de mirar la declaracion y el estado y proponernos un plan para llevarnos del estado actual al que queremos
- Pero la forma en la que llega a eso no siempre es la que queremos o anticipamos.
- Un recreate no es gratuito si el estado actual de la VM o del bucket son importantes. Y muchas veces los cambios que aplicamos parecieran inconsecuentes pero capaz estamos tratando de cambiar un field inmutable, como el tipo de instancia de una VM.
- Al ser inmutable la única forma de aplicarlo es eliminar la VM actual y levantar otra. Pero en ese proceso podríamos perder la configuración del servidor de esa VM por ejemplo. Y eso Terraform no lo hace por nosotros.

Terraform Plan

- El plan depende del state, que es la representación que Terraform tiene de la infraestructura actual.
- Si tenemos muchas personas trabajando en un mismo proyecto tener estados locales que definen que cambios se aplican lleva a problemas rapidamente. Incluso actualizando el estado local con el estado remoto podemos tener condiciones de carrera.
- Por eso se suelen usar configuraciones de estado remoto y locking para evitar dos escrituras en simulteano tanto al state como al estado real.
- Los planes se arman en funcion a un state. Si trato de aplicar un plan armado sobre un state viejo Terraform me va a frenar


Start
Read configuration file
Read state file
Does the resource exist in the state file?
No
Create()
Yes
Read()
Is there a conflict?
No
Is the resource planned for destruction?
No
Update()
Yes
Delete()
Output execution plan
End

Terraform Plan

NUNCA APLIQUEN UN PLAN SIN REVISARLO ANTES